

Escopo:

Escopo do Projeto

O objetivo do projeto é organizar e planejar um novo sistema para a LogiTech Express. O sistema atual apresenta vários problemas, como dados repetidos, informações desorganizadas, lentidão e falta de segurança. Por isso, será criada uma estrutura melhor para o Back-End, deixando o sistema mais organizado e fácil de desenvolver.

Primeiramente, será feita uma organização do banco de dados. Informações como motoristas, veículos, frotas, itens, rotas e entregas serão separadas em tabelas, evitando dados repetidos e facilitando o controle das informações. Também serão definidos os relacionamentos entre os dados para que o banco funcione de forma correta.

O Back-End também será reorganizado para que cada parte do sistema tenha uma função específica. O acesso ao banco de dados, as regras do sistema e as rotas da API ficarão separados. Para isso, serão utilizados o padrão MVC e o Repository, deixando o código mais organizado e fácil de entender e modificar.

A segurança também será melhorada. Senhas, tokens e chaves de acesso não ficarão diretamente no código. Essas informações serão armazenadas no arquivo `.env`, enquanto o `.gitignore` será utilizado para evitar que esses arquivos sejam enviados para o Git. Dessa forma, o risco de vazamento de informações será menor.

Também será definido como o Back-End vai se comunicar com os outros sistemas da empresa. Serão organizadas as rotas da API, os dados que devem ser enviados e as respostas que o sistema deve retornar. Essas informações serão trabalhadas em formato JSON, facilitando a comunicação entre o Back-End e os outros sistemas.

Por fim, será criada uma documentação explicando toda a estrutura planejada, incluindo o banco de dados, o Back-End e a API. A ideia é que qualquer desenvolvedor consiga entender o projeto e continuar o desenvolvimento no futuro. Nesta etapa, não será desenvolvido o sistema completo, nem o Front-End, aplicativo dos motoristas ou a implantação em produção. O foco será deixar toda a estrutura organizada e pronta para a próxima etapa, que será o desenvolvimento do sistema.

Estrutura do Servidor:

routes/

Vai guardar as rotas da API, ou seja, os endereços que o Front-End vai acessar. Por exemplo, uma rota para cadastrar motorista, consultar motorista ou atualizar uma entrega.

controllers/

Vai receber as requisições das rotas e organizar o que precisa ser feito. O Controller não deve ter toda a lógica do sistema nem acessar diretamente o banco.

services/

Vai ficar responsável pelas regras de negócio. Por exemplo, verificar se um motorista já existe antes de permitir um novo cadastro.

repositories/

Vai cuidar da comunicação com o banco de dados. É nessa camada que ficarão as consultas SQL para buscar, cadastrar, alterar ou excluir informações.

models/

Vai representar as entidades utilizadas pelo sistema, como Motorista, Veículo, Frota, Rota e Entrega.

config/

Vai guardar configurações do sistema, principalmente a configuração da conexão com o banco de dados.

middlewares/

Vai conter verificações que precisam acontecer antes de uma requisição continuar, como verificar se o usuário está autenticado.

server.js

Será o arquivo principal responsável por iniciar o servidor e configurar a aplicação.

.env

Vai armazenar informações confidenciais, como usuário, senha e configurações do banco. Essas informações não devem ficar diretamente no código.

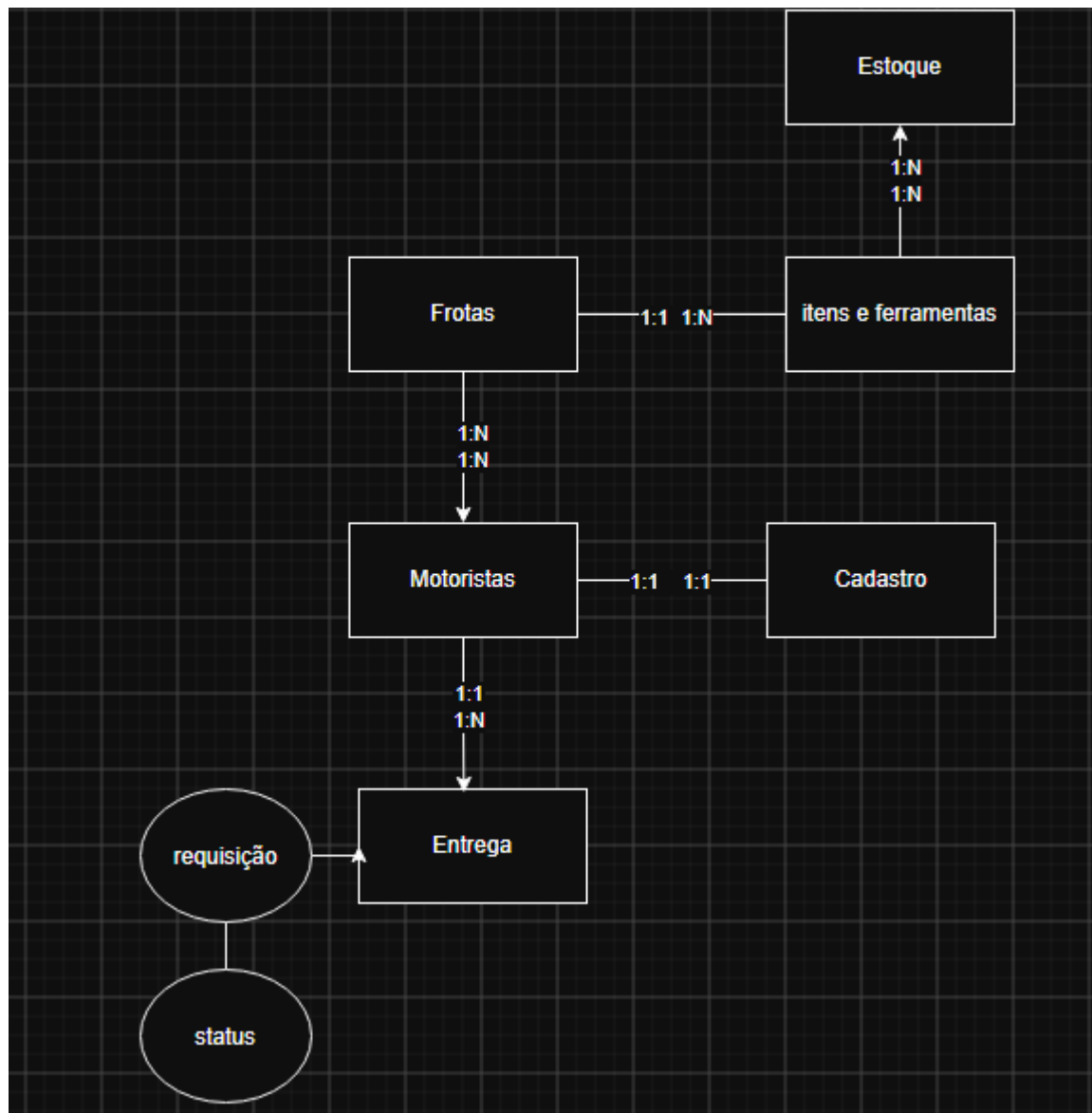
.gitignore

Vai indicar arquivos que não devem ser enviados para o Git, principalmente o `.env`.

TESTS.md

Será o documento que vocês acabaram de fazer, contendo os cenários e validações que deverão ser testados na próxima UC.

Desenho do banco (DER):



Plano de testes documental

Testes de Motoristas

Cadastro de motorista válido

- Verificar se um motorista pode ser cadastrado quando todos os dados obrigatórios estiverem preenchidos corretamente.

Resultado esperado: O motorista deve ser cadastrado.

Cadastro com campos obrigatórios vazios

- Tentar cadastrar um motorista deixando um ou mais campos obrigatórios vazios.

Resultado esperado: O sistema deve impedir o cadastro e informar quais dados estão faltando.

Cadastro de motorista duplicado

- Tentar cadastrar um motorista utilizando um CPF que já está cadastrado.

Resultado esperado: O sistema deve impedir a duplicidade e informar que o motorista já existe.

Consulta de motorista

- Consultar um motorista utilizando seu identificador.

Resultado esperado: O sistema deve retornar corretamente os dados do motorista.

Motorista inexistente

- Tentar consultar um motorista que não existe no banco de dados.

Resultado esperado: O sistema deve informar que o motorista não foi encontrado.

Atualização de motorista

- Alterar os dados de um motorista já cadastrado.

Resultado esperado: Os novos dados devem ser salvos.

Exclusão de motorista

- Excluir um motorista existente.

Resultado esperado: O registro deve ser removido ou tratado de acordo com as regras definidas para o sistema.

Testes de Frotas e Veículos

Cadastro de veículo válido

- Verificar se um veículo pode ser cadastrado com todos os dados necessários.

Resultado esperado: O veículo deve ser cadastrado corretamente.

Cadastro de veículo duplicado

- Tentar cadastrar um veículo utilizando uma placa que já existe.

Resultado esperado: O sistema deve impedir o cadastro duplicado.

Associação de motorista e veículo

- Associar um motorista a um veículo.

Resultado esperado: A associação deve ser registrada corretamente no banco.

Dados inválidos do veículo

- Tentar cadastrar um veículo com informações inválidas ou campos obrigatórios vazios.

Resultado esperado: O sistema deve rejeitar os dados e informar o erro.

Testes de Ferramentas e Itens

Cadastro de ferramenta

- Cadastrar uma ferramenta vinculada a uma frota.

Resultado esperado: A ferramenta deve ser registrada corretamente.

Consulta de ferramentas

- Consultar as ferramentas pertencentes a uma determinada frota.

Resultado esperado: O sistema deve retornar os itens relacionados à frota.

Quantidade inválida

- Tentar cadastrar uma quantidade negativa ou inválida de ferramentas.

Resultado esperado: O sistema deve impedir o cadastro.

Ferramenta duplicada

- Tentar cadastrar novamente uma ferramenta que já existe para a mesma frota.

Resultado esperado: O sistema deve tratar a duplicidade de acordo com as regras definidas.

Testes de Rotas

Criação de rota

- Criar uma rota informando os dados necessários.

Resultado esperado: A rota deve ser criada com sucesso.

Rota sem motorista

- Tentar criar uma rota sem informar o motorista quando essa informação for obrigatória.

Resultado esperado: O sistema deve rejeitar a solicitação.

Motorista inexistente

- Tentar associar uma rota a um motorista que não existe.

Resultado esperado: O sistema deve impedir a associação.

Atualização de rota

- Alterar os dados de uma rota existente.

Resultado esperado: Os dados devem ser atualizados corretamente.

Testes de Entregas

Cadastro de entrega

- Cadastrar uma entrega com todos os dados obrigatórios.

Resultado esperado: A entrega deve ser registrada corretamente.

Entrega com dados incompletos

- Tentar cadastrar uma entrega sem informações obrigatórias.

Resultado esperado: O sistema deve impedir o cadastro e informar o erro.

Atualização do status da entrega

- Alterar o status de uma entrega para um status permitido.

Resultado esperado: O novo status deve ser salvo corretamente.

Status inválido

- Tentar atualizar uma entrega utilizando um status que não existe nas regras do sistema.

Resultado esperado: O sistema deve rejeitar a alteração.

Entrega inexistente

- Tentar consultar ou alterar uma entrega que não existe.

Resultado esperado: O sistema deve informar que a entrega não foi encontrada.

Testes da API

Requisição válida

- Enviar uma requisição com todos os dados no formato esperado.

Resultado esperado: A API deve processar a solicitação e retornar a resposta correta.

JSON inválido

- Enviar dados em formato diferente do esperado pela API.

Resultado esperado: A API deve rejeitar a requisição e retornar uma mensagem de erro.

Endpoint inexistente

- Tentar acessar uma rota da API que não existe.

Resultado esperado: A API deve retornar erro informando que a rota não foi encontrada.

Método HTTP incorreto

- Utilizar um método HTTP diferente daquele definido para determinado endpoint.

Resultado esperado: A API deve rejeitar a requisição ou informar que o método não é permitido.

Testes de Autenticação e Segurança

Login com dados corretos

- Realizar login utilizando credenciais válidas.

Resultado esperado: O acesso deve ser permitido.

Login com senha incorreta

- Tentar realizar login utilizando uma senha incorreta.

Resultado esperado: O acesso deve ser negado.

Login com usuário inexistente

- Tentar realizar login com um e-mail ou usuário que não está cadastrado.

Resultado esperado: O sistema deve negar o acesso.

Acesso sem autenticação

- Tentar acessar uma rota protegida sem realizar login.

Resultado esperado: O sistema deve bloquear o acesso.

Proteção de senha

- Verificar se as senhas dos usuários não são retornadas nas respostas da API.

Resultado esperado: Informações de senha não devem ser expostas.

Proteção de credenciais

- Verificar se senhas, tokens e chaves de API não estão diretamente no código-fonte.

Resultado esperado: Informações confidenciais devem estar protegidas por variáveis de ambiente.

Testes de Erros

Erro de conexão com o banco

- Simular uma situação em que o banco de dados não esteja disponível.

Resultado esperado: O sistema deve apresentar uma mensagem de erro adequada sem travar completamente a aplicação.

Registro não encontrado

- Solicitar informações de um registro inexistente.

Resultado esperado: A API deve retornar uma resposta indicando que o registro não foi encontrado.

Dados inválidos

- Enviar informações fora do padrão esperado.

Resultado esperado: A API deve rejeitar os dados e informar o motivo do erro.

Validações Gerais

- Todos os campos obrigatórios deverão ser validados antes de serem enviados ao banco de dados. Os dados deverão possuir o formato correto e os relacionamentos entre as tabelas deverão ser respeitados.
- O sistema também deverá impedir registros duplicados quando houver uma regra que exija unicidade, além de impedir valores inválidos ou incompatíveis com os tipos definidos no banco de dados.

Resultado dos Testes

- Na próxima Unidade Curricular, cada cenário deverá ser executado e registrado como:
 - "Aprovado:" quando o sistema apresentar o resultado esperado.
 - "Reprovado:" quando o sistema apresenta um comportamento diferente do esperado.

Normalização:

ID	Pilar	Problema atual	Solução proposta	Padrão	Resultado esperado
1	Saneamento e Modelagem de Dados	Motoristas cadastrados em duplicidade.	Criar um cadastro único para cada motorista e utilizar identificadores únicos.	Normalização, PK e UNIQUE .	Eliminar duplicidades e garantir integridade.
2	Saneamento e Modelagem de Dados.	Itens e ferramentas armazenados em uma única string,	Criar tabelas específicas para	1FN, 2FN e 3FN.	Permitir controle de estoque e auditoria.

		separados por vírgulas.	itens, ferramentas e estoque.		
3	Saneamento e Modelagem de Dados.	Dados redundantes e sem organização adequada.	Separar os dados em entidades independentes e relacioná-las por chaves.	Modelo ER, PK e FK.	Reduzir redundância e facilitar manutenção.
4	Saneamento e Modelagem de Dados.	Falta de integridade entre motoristas, veículos e frotas.	Criar relacionamentos entre as entidades.	Banco de dados relacional.	Garantir consistência dos dados.
5	Saneamento e Modelagem de Dados	Dificuldade para controlar entregas e rotas.	Criar entidades próprias para entregas e rotas.	Modelagem relacional.	Melhor rastreamento das operações.
6	Arquitetura Limpa e Escalável.	Requisições de atualização de entrega chegam despadronizadas.	Definir endpoints e formatos padronizados para a API.	REST API + JSON.	Comunicação consistente entre sistemas.
7	Arquitetura Limpa e Escalável.	Código de rotas, banco de dados e respostas está misturado.	Separar o sistema em camadas.	MVC.	Melhor organização do Back-End.
8	Arquitetura Limpa e Escalável.	Regras de negócio estão misturadas com outras responsabilidades.	Criar uma camada específica para as regras de negócio.	Service Layer.	Facilitar manutenção e testes.
9	Arquitetura Limpa e Escalável.	Acesso ao banco ocorre junto com a lógica da aplicação.	Criar uma camada exclusiva para comunicação com o banco.	Repository Pattern.	Reduzir o acoplamento e facilitar alterações.
10	Arquitetura Limpa e Escalável.	Servidores apresentam sobrecarga e as telas travam.	Organizar responsabilidades e otimizar o processamento das requisições.	Arquitetura em camadas.	Melhor desempenho e escalabilidade.
11	Arquitetura Limpa e Escalável.	Mensagens de erro são genéricas.	Criar tratamento centralizado e padronizado de erros.	Middleware + HTTP Status Codes.	Erros mais claros e úteis.
12	Segurança e Contratos.	Senhas e chaves de acesso estão expostas no código.	Retirar credenciais do código e utilizar	.env	Proteger informações confidenciais.

			variáveis de ambiente.		
13	Segurança e Contratos.	Tokens de APIs externas estão expostos.	Armazenar tokens em variáveis de ambiente e configurar sua proteção.	.env + .gitignore	Reduzir risco de vazamento.
14	Segurança e Contratos	Houve tentativa de invasão e vazamento de dados.	Revisar credenciais, aplicar autenticação e controlar acesso aos recursos.	Autenticação + autorização.	Aumentar a segurança da aplicação.
15	Segurança e Contratos.	Não existe um padrão definido para os dados enviados à API.	Definir contratos de requisição e resposta.	JSON + DTO.	Facilitar integração com qualquer Front-End.
16	Segurança e Contratos	Dados recebidos pela API podem chegar inválidos ou incompletos.	Implementar validação dos dados recebidos.	DTO + Validation.	Evitar dados incorretos no sistema.

<i>Entidade</i>	<i>Principais dados</i>	<i>Função</i>
Motorista	ID, nome, CPF, telefone, status	Armazenar os dados dos motoristas.
Frota	ID, nome, localização, status	Organizar os veículos da empresa.
Veículo	ID, placa, modelo, tipo, status	Controlar os veículos utilizados.
Item	ID, nome, tipo, quantidade	Controlar ferramentas e itens.
Frota_Item	ID da frota, ID do item, quantidade	Relacionar frotas e itens.
Entrega	ID, origem, destino, prioridade, status	Controlar as entregas.
Rota	ID, origem, destino, distância	Armazenar informações das rotas.

<i>CAMADA</i>	<i>RESPONSABILIDADE</i>
Routes	Define os endpoints da API
Controller	Recebe as requisições e envie as respostas
Service	Contém as regras de negócio
Repository	Faz a comunicação com o banco de dados
Database	Armazena os dados de aplicação
Middleware	Trata autenticação, validação e erros
DTO	Define o formato dos dados recebidos e enviados

